

1

1. (不定项选择, 2 分) 以下描述, 哪些属于 ISA 规定的内容:

- A. 有 15 个通用寄存器, 每个寄存器 64 位
- B. 有 3 个条件码, 分别是 ZF、SF、OF
- C. 过程调用指令 (call) 会将下一条指令地址压入栈中
- D. 过程调用的前 2 个参数放在寄存器 rdi、rsi
- E. 过程调用返回值放在寄存器 rax
- F. 内存是字节寻址的, 字节序采用小端法
- G. 访存指令支持寄存器加偏移的寻址模式

答案: ABCFG

ISA规定了各种状态单元, 指令集, 编码, 编程规范和异常事件处理。

有哪些寄存器、寄存器宽度

有哪些指令、指令的行为

条件码

寻址方式

内存字节序

指令编码规则

DE是由ABI规定的: 编译器与操作系统之间的约定

2

2. (2 分) Y86-64 中的这两条指令:

```
rmovq rA, D(rB)
```

```
mrmovq D(rB), rA
```

第 2 条指令为什么不设计成 `mrmovq D(rA), rB`? 这样都是 rA 在前 rB 在后, 看起来更加规整。

答:

```

// Select input A to ALU
u64 aluA = [
    E.icode in { CMOVX, OPQ } : E.valA;
    E.icode in { IRMOVQ, RMMOVQ, MRMOVQ } : E.valC;
    E.icode in { CALL, PUSHQ } : NEG_8;
    E.icode in { RET, POPQ } : 8;
    // Other instructions don't need ALU
];

// Select input B to ALU
u64 aluB = [
    E.icode in { RMMOVQ, MRMOVQ, OPQ, CALL, PUSHQ, RET, POPQ } : E.valB;
    E.icode in { CMOVX, IRMOVQ } : 0;
    // Other instructions don't need ALU
];

```

alu单元，aluA 可以是valA,valC,8或-8

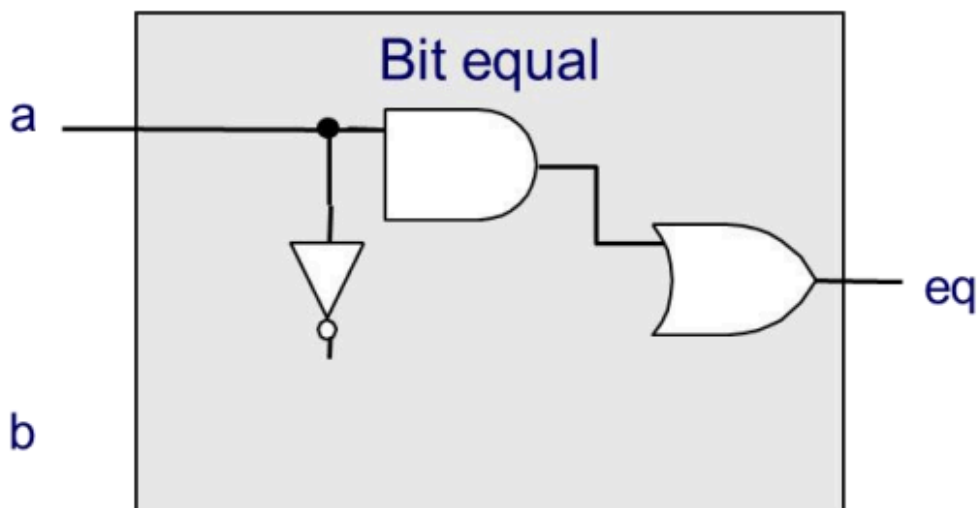
aluB 是0或valB——来自rB

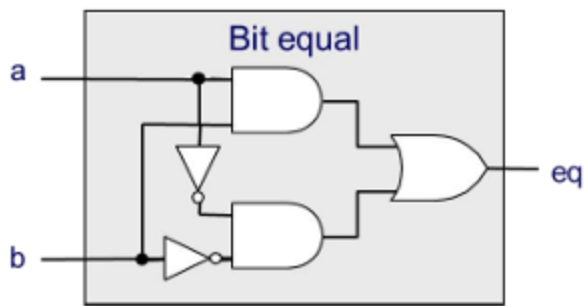
硬件电路上通过rB计算出地址，这样使电路更为简单

3

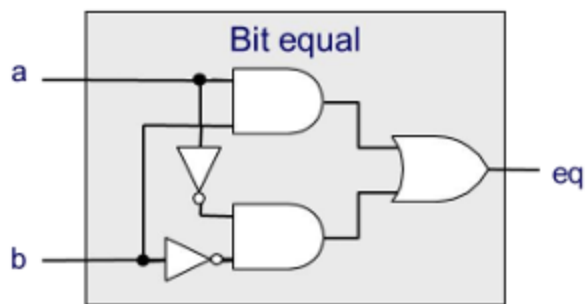
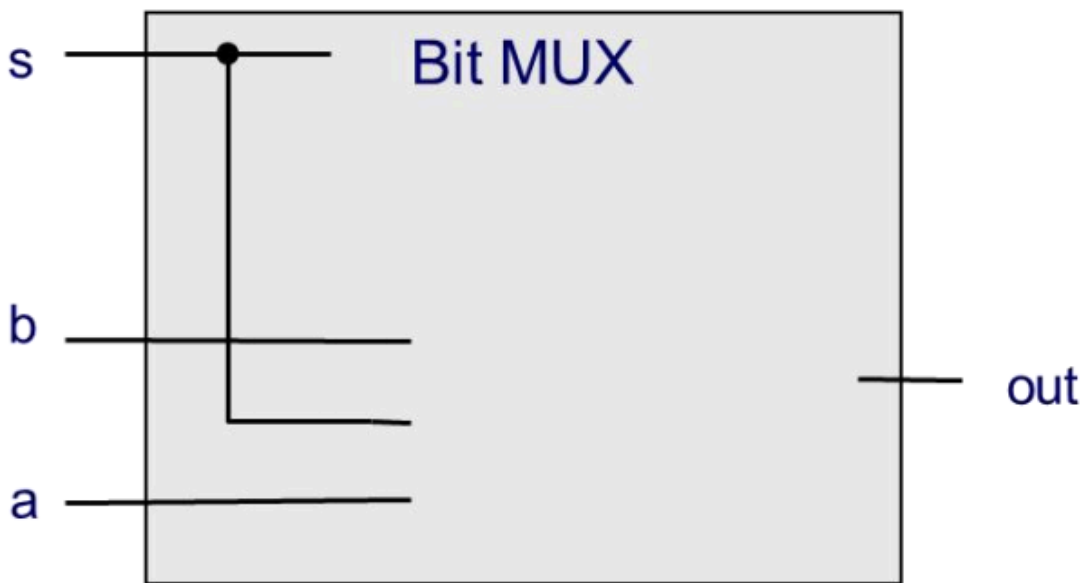
考点：组合电路

(1) $\text{bool eq} = (a \& \& b) \mid \mid (!a \& \& !b)$



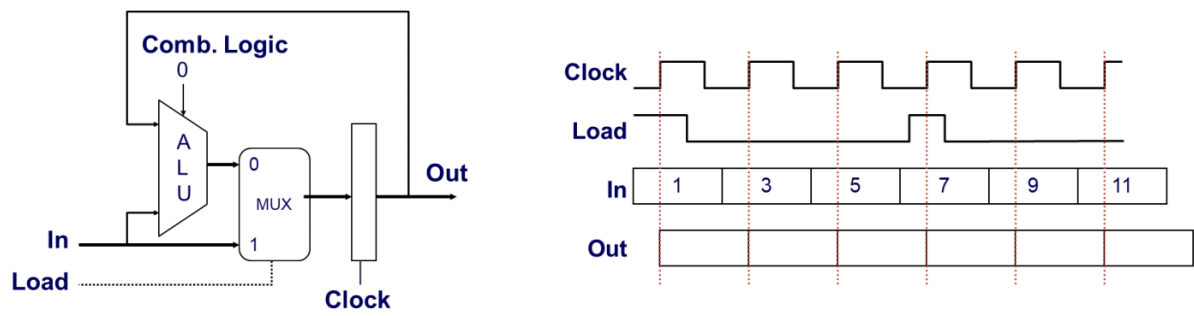


(2) `bool out = (s&&a) || (!s&&b)`



4

4. （2 分）根据下面左边电路图，补全右边时序图里 Out 信号的值（In/Out 均用 10 进制表示）



考点：

多路复用器和时钟

时钟：

当时钟是低点位时，寄存器的输出不变，当时钟变成高电位，输入信号就加载到寄存器中。

多路复用器：

由输入位决定选择哪一个值，这里的输入位是Load

5

考察指令的处理

5. (8 分) Y86-64 的 popq 指令编码格式如下:



根据 popq 指令的操作, 补全下表 (共需填 8 行 9 空)

	popq rA
Fetch	icode:ifun ← M ₁ [PC]
	rA:rB ← M ₁ []
	valP ← PC+2
Decode	valA ← R[]
	valB ← R[]
Execute	valE ←() + ()
Memory	() ← M ₈ [valA]
Write back	R[] ← valE
	R[] ← valM
PC update	PC ← ()

PC+1
%rsp
%rsp
valB 8
valM
%rsp
rA
valP

6

考察SEQ

	OPq rA, rB
Execute	$valE \leftarrow valB \text{ OP } valA$
	cmovXX rA, rB
Execute	$valE \leftarrow 0 + valA$
	rmmovq rA, D(rB)
Execute	$valE \leftarrow valB + valC$
	popq rA
Execute	$valE \leftarrow valB + 8$
	jXX Dest
Execute	
	call Dest
Execute	$valE \leftarrow valB + (\quad)$
	ret
Execute	$valE \leftarrow valB + (\quad)$

```
int aluA = [
    icode in { IRRMOVQ,      } : valA;
    icode in { IIRMOVQ,      , IMRMVQ } : valC;
    icode in {                , IPUSHQ } :      ;
    icode in {                , IPOPQ  } :      ;
    # Other instructions don't need ALU

];
```

-8

8

IOPQ

IRMMOVQ

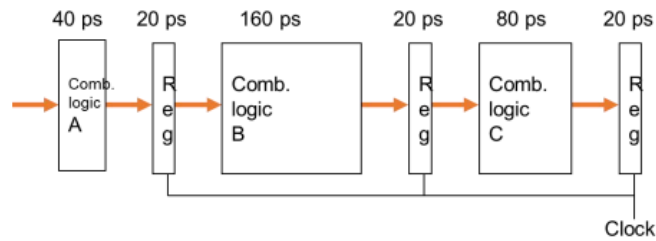
ICALL -8

IRET 8

7

考察流水线的延迟计算

7. (4 分) 如下图所示的三级流水线结构, 单条指令的延迟 (delay) 是_____ps, 流水线满负荷运转时, 每_____ps 完成一条指令。如果能理想化地进一步细分流水级 (不跨现有流水级重排电路、也不过度细分), 恰好划分成均衡流水线, 那单条指令的延迟 (delay) 是_____ps, 流水线满负荷运转时, 每_____ps 完成一条指令。



单条指令的延迟: 将所有阶段的延迟和寄存器的延迟加起来

流水线满负荷运转: 将寄存器延迟和用时最长的阶段的延迟加起来

恰好划分成均衡流水线:

由于题目说不跨现有流水线, 不过度细分, 注意到原流水线的最小阶段延迟是40ps, 那么就是将每个阶段延迟都变成40ps。则需要将B阶段拆成四部分, 将C阶段拆成两部分。

答案:

340 180 420 60



8. (6 分) 阅读下面这段 Y86-64 汇编代码

- ① `irmovq $1,%rax`
- ② `irmovq $2,%rax`
- ③ `irmovq $3,%rdx`
- ④ `addq %rax,%rdx`

4

当这段代码在 PIPE 处理器上运行时，当 `addq` 指令在流水级_____阶段时，会检测到数据冒险。此时运用了数据前递技术，其中在流水级_____阶段的指令前递数据_____作为操作数 `%rax` 的值。

答案：

D M 2

`addq` 指令在 D 阶段需要读取寄存器 `%rax` 和 `%rdx` 的值，而由于指令 2, 3 之前会修改这两个寄存器的值，所有会在 D 阶段发生数据冒险。

指令 2 修改了寄存器 `%rax` 的值，在此时指令 2 处于 M 阶段，传递数据 2. 这里转发的是 M_valE。

_____ 作为操作数 `%rax` 的值。

9. (4 分) 写出一段存在 Load-use 相关的 Y86-64 汇编代码 (要求代码简洁明了)，并说明能否用数据前递技术解决，为什么？

答：

```

mrmovq 0(%rbp), %rax    # load: 从内存读到 %rax
addq    %rax, %rdx       # use: 下一条立即使用 %rax → load-use hazard

```


不能

因为 后一条指令在译码阶段就需要使用到寄存器的值，此时前一条指令在E阶段，而到M阶段才能读出需要的值。

10

10. (2 分) 写出一段存在 Load-use 相关和 ret 组合的 Y86-64 汇编代码 (要求代码简洁明了)。

答:

```
mrmovq 0(%rsp), %rax # load (从栈加载到 %rax)
ret # 紧跟着使用 %rax (ret 需要读 %rsp)
# → load-use + ret 组合冒险
```